

EE272 Homework 4

HLS Design

Due: Feb 5, 2026, 11:59 pm

Released: Jan 29, 2026

1 High Level Synthesis with Catapult

In this homework, you will generate an RTL design of your CNN accelerator using high level synthesis (HLS), and perform behavioural verification. Your goal is to generate an RTL design similar to the one you painstakingly wrote in homework 3, only this time by writing it in C++ and guiding an HLS tool, using compiler directives, to produce the design that you want. It is usually relatively easy to produce a functionally correct design with HLS, because HLS takes care of generating all the error-prone control logic in your chip (for example, your complex top-level convolution controller). It is, however, much harder to produce an implementation that has high performance, because you have to give the scheduler enough hints about what to unroll and pipeline. In homework 5, you will focus on optimizing your design to have high performance. Since you hand wrote the RTL in homework 2 and 3, you know the architecture you are looking for and its performance (in terms of number of cycles) very well. Always keep your homework 3 architecture in mind while using HLS, and try and see if HLS is producing the same (or better!) design. This will require you to understand deeply how HLS translates C++ constructs into hardware. Use the two HLS lectures for guidance. **HLS tools have a steep learning curve, so this homework will need significant time. Please start early.**

The high level synthesis tool we will use is Catapult from Siemens EDA. It takes HLS C++ code and specified pragmas and directives, and generates RTL Verilog code. Writing HLS code is very similar to writing C++ code, but in a more restricted way. The code structures and styles have to satisfy certain conditions in order to generate reasonable and efficient designs. Pragmas and directives are used for specifying resource type (whether a memory is implemented using registers or SRAM), loop pipelining and unrolling, etc.

2 Getting Started

The starter code can be found at this [code repo](https://code.stanford.edu/ee272/dnn-accelerator-hls-unoptimized.git) (<https://code.stanford.edu/ee272/dnn-accelerator-hls-unoptimized.git>). To get started, log in to the FarmShare machine and run the Singularity image as before, and then clone the homework repository:

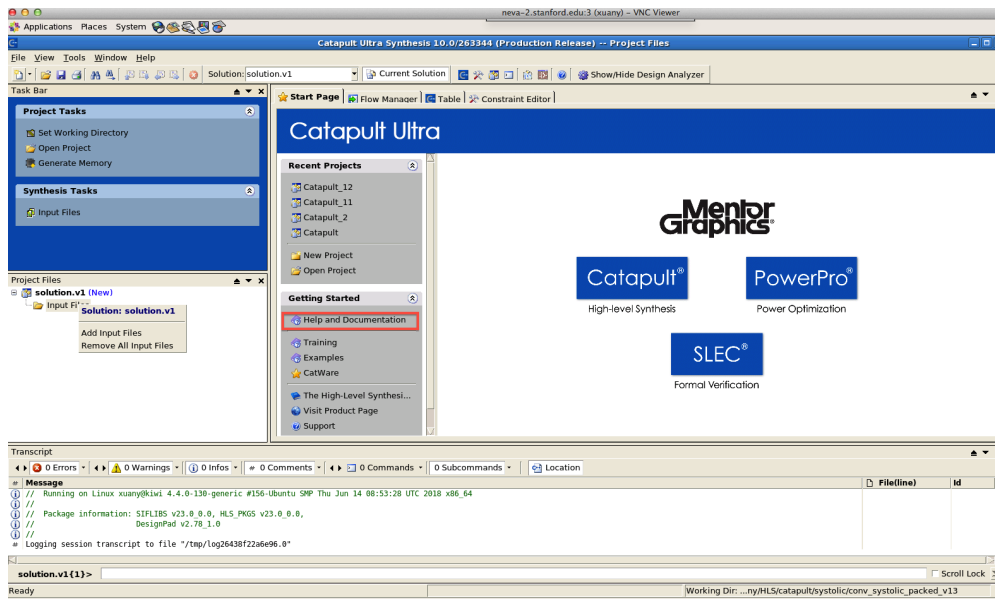


Figure 1: Catapult GUI.

```
$ git clone
https://code.stanford.edu/ee272/dnn-accelerator-hls-unoptimized.git
```

Then make sure you are at the top level of this code repo before running any commands:

```
$ cd dnn-accelerator-hls-unoptimized
```

Next, set up the environment variables by sourcing the setup file:

```
$ source setenv.csh
```

3 Catapult Documentation

Catapult has a **very steep** learning curve. You will find the following resources helpful while using Catapult to complete this homework.

- HLS Bluebook: Use this to get a general understanding of how to write HLS code. It is at the following location on caddy machines: /cad/mentor/2019.11/Catapult_Synthesis_10.4b-841621/Mgc_home/shared/pdffdocs/hls_bluebook.pdf
- Catapult User Reference: For command related questions see /cad/mentor/2019.11/Catapult_Synthesis_10.4b-841621/Mgc_home/shared/pdffdocs/catapult_useref.pdf on caddy machines.

- Catapult User Manual: This is the GUI version of the User Reference above. It can be accessed by clicking **Help and Documentation** in Catapult GUI as shown in Figure 1.
- Edge Detection HLS Walkthrough:
https://www.youtube.com/channel/UCJMnUQKLcNNQ_KrD97_sU6w/playlists. This series provides a step-by-step walk-through of what is needed to take a C++ floating-point algorithm all the way to optimized RTL using Catapult synthesis on an edge detection design. The sources for the design are up on GitHub at https://github.com/hlslibs/hls_tutorials/tree/master/WalkThroughs/EdgeDetect/src, so you can download the examples and follow along.

4 Catapult Tutorial

The following tutorial will introduce some common errors when interacting with the types and channels used in this assignment. The following steps will help you get the tutorial set up. First, copy the repository source files into a location in your directory, called <folder>.

```
$ cp -rf /cad/mentor/2022.2/Mgc_home/shared/training/basic/cxx <folder>
```

Go into this directory and give write permissions to the files and folders, such as cxx, lab1, and lab2. If you encounter an error later in this tutorial, it is likely related to the file write permissions. As you go through the tutorial, add write permissions to files as necessary. The following is an example of how to add write permissions.

```
$ cd <folder>
$ chmod +w ./cxx
```

Set the location of the MGC files. This variable will be used in the Makefile for Lab 1.

```
$ setenv MGC_HOME /cad/mentor/2022.2/Mgc_home
```

Open the Makefile for Lab 1 and set the temporary directory variable (TDIR) to run in the current directory. If you run the terminal command from within Lab 1, use the following. Otherwise, you can hardcode the path to <folder>/cxx/lab1.

```
TDIR ?= $(shell pwd)
```

Complete the tutorial for Lab 1 as posted on Canvas. Note that provided line numbers may not exactly match. Refer to the homework outline for screenshots to submit for this portion of the assignment. Other tutorials are available in this folder for your reference.

5 Code Organization

The code organization in `dnn-accelerator-hls-unoptimized` folder is shown below, along with what each file contains.

```

src
    boost                                // C++ macro library you shouldn't touch

    Conv.cpp                            // Top level HLS module
    InputDoubleBuffer.h                 // Input double buffer
    InputDoubleBufferTb.cpp             // Testbench for InputDoubleBuffer
    WeightDoubleBuffer.h                // Weight double buffer
    WeightDoubleBufferTb.cpp            // Testbench for WeightDoubleBuffer
    Fifo.h                             // Templated FIFO for skewing
    ProcessingElement.h                 // Combinational MAC
    SystolicArray.h
    SystolicArrayCore.h                // Contains skewing FIFOs, combinational
                                        // MAC and registers array,
                                        // accumulation buffer

    Deserializer.h                     // Deserializes the layer parameters
    Serializer.h                       // Serializes the outputs from the
                                        // accumulation buffer

    conv.h                             // Contains top level instantiation
                                        // parameters and data structures

    ConvTb.cpp                          // Testbench for Conv.cpp
    conv_gold.cpp                       // Gold model you wrote earlier
    conv_gold_tiled.cpp                 // Tiled gold model you wrote earlier
    conv_tb_params.h                   // Layer parameters used for configuring
                                        // the testbench

scripts                                // Catapult scripts
    buffer.mk                           // Makefile for buffer tests
    common.tcl                          // HLS options common to all modules
    Conv.tcl                            // Module specific Catapult scripts
    InputDoubleBuffer.tcl
    WeightDoubleBuffer.tcl
    ProcessingElement.tcl
    SystolicArrayCore.tcl
    generic_libraries.tcl              // Defines the technology
    set_libraries.tcl                  // Uses the above library
    run_c_test.tcl                     // Runs tests in C++
    run_rtl_test.tcl                   // Runs tests on the generated RTL and
                                        // opens the Catapult waveform viewer

    run_rtl_test_no_gui.tcl

layers
    compare                             // Holds gold vectors for buffer tests
    Makefile
    autograder.py
    setenv.csh

```

6 Testbench

For this homework, we give you the testbench. This is defined in `src/ConvTb.cpp`. It would be helpful for you to read the testbench to understand in what order the data is streamed in and out of your accelerator. To run the testbench on your C++ code, do:

```
$ make c_test
```

or

```
$ make c_fast_test
```

The difference is that `c_test` target compiles your code with Catapult. It makes sure that your C code will be synthesizable to RTL before running the C simulation. This is good for verification, but it takes a lot of time. The `c_fast_test` runs your code faster but does not check if your C code is synthesizable. You should use `c_fast_test` when you are sure your code will synthesize and you need to iterate on your design. In the end, your code must compile through the `c_test` target.

To generate the RTL and then run the testbench on the generated RTL, do:

```
$ make rtl_test
```

You should read the `Makefile` and `buffer.mk` to see what commands to run for running any of the sub steps individually.

7 Implementation

In the starter code, we have created the different blocks that make up the design. These are in the various files in the `src` folder. The Code Organization section summarizes what each file contains.

- The hierarchy of the design is:

Conv (`Conv.cpp`)

- InputDoubleBuffer (`InputDoubleBuffer.h`)
 - * InputBankWriter
 - * InputBankReader
- WeightDoubleBuffer (`WeightDoubleBuffer.h`)
 - * WeightBankWriter
 - * WeightBankReader
- SystolicArrayWrapper (`SystolicArray.h`)
 - * SystolicArrayLooper

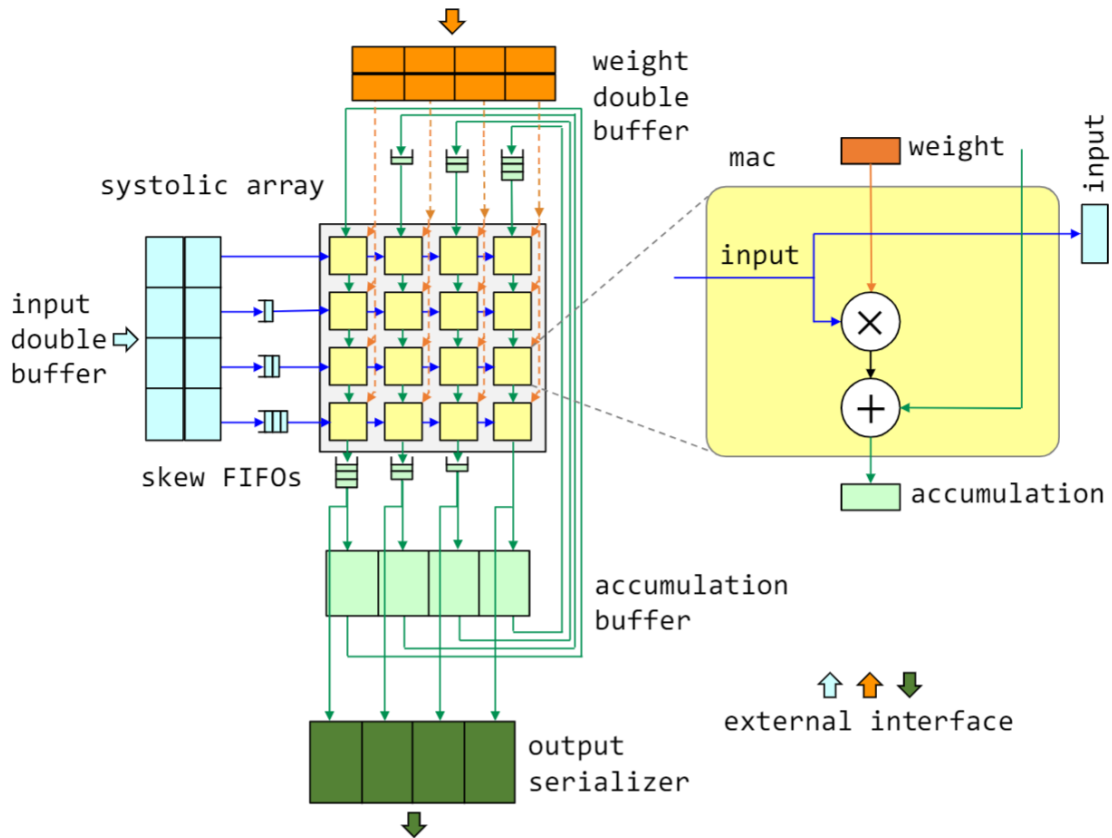


Figure 2: CNN accelerator.

```
* SystolicArrayCore (SystolicArrayCore.h)
  · ProcessingElement (ProcessingElement.h)
```

- You should start by reading `conv.h` that defines the accelerator parameters and datatypes. The parameter names roughly correspond to what they were called in homework 2 and 3. For debugging, use smaller parameter values like a 4×4 array.

Note that when you get to the synthesis step, if you changed the default parameters, the scripts will need to be slightly modified, as they are written for the default parameters defined in `conv.h`. For example, if you changed the `ARRAY_DIMENSION` in the C++ code, you will have to change the `ARRAY_DIMENSION` value in `scripts/common.tcl` as well.

- Then read `Conv.cpp` to see how the datatypes are used, and how the major sub-modules are instantiated and connected. You do not have to add any code in this file, but read it to make sure you understand what is going on.
- Notice that `params channels` are used as control flow for the accelerator. Remember that `ac_channel` reads are blocking, so we can use them for control flow. In

the accelerator, there are a few instances where we will write params multiple times such that a module will run as many times as we write params to it.

7.1 Double Buffers

- **Your first task is to implement the weight double buffer and then the input double buffer.** We have provided you all the interfaces, and the files indicate the places where you have to fill in code. In each buffer, you must implement a writer and reader, which run independently of each other. After implementing the modules, you can run a `c_test` with following example commands:

```
$ python3 autograder.py test_c_weight_test -l all
$ python3 autograder.py test_c_input_test -l all
```

- **Before implementing double buffers go through the HLS lecture and read Chapter 8.1.7 in the HLS Bluebook**— the code needs to be written in a certain style for Catapult to infer a double buffer correctly.

7.2 Systolic Array

- **Next, implement the processing element and the systolic array.** The processing element is combinational unlike the one in homework 3, meaning the input, weight and output registers are instantiated **outside** the processing element in the systolic array core. The systolic array core reads from the weight double buffer and writes to the weight registers. (Note: In this homework the weights are **not** skewed. You need to load the weights to weight registers properly so that they are ready for PEs to use.) It reads inputs from the input double buffer and feeds into the skew FIFOs, which feed the PE array. (Note: In your previous homework you had skew registers, but this homework you will use skew FIFOs, which are already created for you (see Figure 2). You will need to hook these up. It is important to remember that in this assignment when we talk about FIFOs, we are referring to skew FIFOs.) The outputs from the PE array are skewed and written to the accumulation buffer as before, and they are read out and fed back into the PE array from the top to accumulate across FX, FY and IC dimensions. Before implementing, answer the questions below. The answers should be in terms of configuration parameters (tiling, filter size, etc.) with some absolute numbers.

1. How many cycles does the systolic array run to produce a valid output tile?
2. When will the first valid output pixel be output to the output skew registers?
3. How many weight lines (OC0 weights) will be loaded during the tile computation?
4. How many input pixel lines (IC0 pixels) will be loaded during the tile computation?
5. How many partial sums (OC0 partial sums) will be loaded during the tile computation?

6. When should we load 0 instead of the output of the accumulation buffer for partial sums into the systolic array?
- The systolic array source code is contained in `SystolicArray.h` and `SystolicArrayCore.h`. The former defines a looper which generates the loop indices that are fed to the systolic array core. It determines the number of times it runs the `run` function of `SystolicArrayCore`. Notice that `paramsChannel` is an `ac_channel`, so in `SystolicArrayLooper` function you will have to write to it as many times as you write to `LoopIndices`, despite the fact that the values are always the same.
 - In this homework the two banks of the output buffer are separated (see Figure 2). You will implement the bank used for accumulation in `SystolicArrayCore`. You will not need to implement the bank that captures the output that is being sent out of the accelerator. This has been done for you. Make sure to write the outputs to the output double buffer in `SystolicArrayCore`.

7.3 Compilation and Debugging

- Once you have an implementation, run the C test first to check that the C code you have written is functional. Debugging this is like debugging normal C code. If you would like to use gdb, the binary file is located at:
`./test/Conv.v1/scverify/orig_cxx_osci/scverify_top`
- Another way to debug the design is to edit the `ConvTb.cpp` to give the array predictable inputs and weights and print intermediate steps of the design to verify operation. This will be a very useful strategy for debugging the C code part of the design process. You can change `rand_init` to 0 and test with pre-defined inputs and weights. You also can also log the PE state into a file for each iteration by setting `HLS_DEBUG` to 1 within `SystolicArrayCore.h`.
- **Tips for debugging empty `ac_channels`.** First, use gdb and backtrace to locate where the channel is failing in the code. You can print out its contents and size with `[]` and `size()` operators. Try to analytically calculate the size and compare it with what you measured to trace the issue.
- In many circumstances, the empty `ac_channel` might be caused by incorrect loop bounds, such that the number of attempted reads and writes mismatches.
- Once you have a functional design, you will now need to modify the build scripts to make sure that the tool synthesizes your design into what you are expecting. Read through the scripts in `scripts/` and understand what needs to be done in each file. Since you need to fill in commands, it might be helpful to open the GUI and see what commands should go in the script. Hint: For the double buffers you will need to set the `WORD_WIDTH` for the reader temporary variable and `din` and the writer temporary variable and `dout`. You will also need to set `WORD_WIDTH` and `STAGE_REPLICATION` for `mem` and `mem:cns`, respectively. Below is a screenshot showing you how to navigate to `mem`. You will need to follow similar steps

for `SystolicArrayCore.tcl`. Specifically, you should set the correct `INTERLEAVE` and/or `BLOCK_SIZE` for the `accumulation_buffer` (refer to User Reference), make sure the registers map to registers and not memories, set a register threshold, and ignore memory dependencies that the tool assumes are occurring. Try building the different blocks and when the script stops at the unimplemented section, open the GUI:

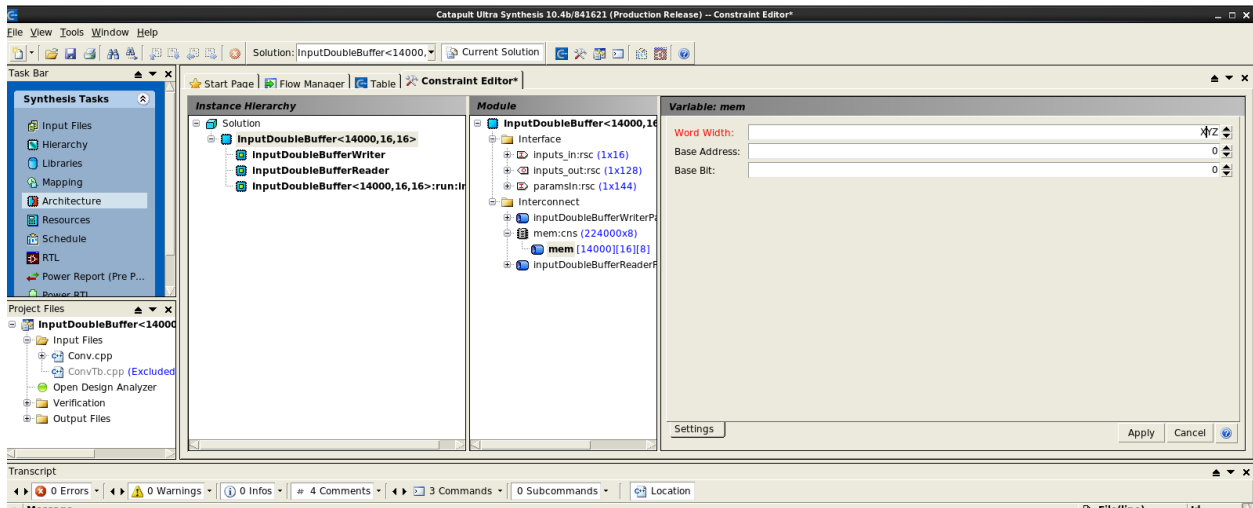


Figure 3: Catapult GUI.

```
$ make InputDoubleBuffer
$ make gui
```

- You will also need to look at the schedule for each block. Click on the Schedule tab to open up the schedule for a block. Select between the different blocks in your design in the dropdown as shown in Figure 4. Since optimization is not done in this homework, your throughput will likely be bad. In the next assignment your goal will be to optimize the performance of your design. (Get a head start by understanding the Catapult scheduler early!)
- Follow the same procedure until you have completed the build script for every single block:

```
$ make WeightDoubleBuffer
$ make gui
```

```
$ make ProcessingElement
$ make gui
```

```
$ make SystolicArrayCore
$ make gui
```

When running SystolicArrayCore scheduling you may run into “could not schedule even with unlimited resources” error. This is because of the false memory dependencies issue that we covered in the lecture. To investigate which memory operations are causing the issue, in the GUI click **Schedule** to start scheduling. It will fail and display the error messages in red. You can then figure out the operations that you need to set `ignore_memory_precedences` for. Hint: These commands are hard to get a hang of so here are some tips. Read page 838 in Catapult User Reference. You can determine operation pathnames from the Gantt chart by right-clicking on an operation and choosing Copy Operation Path to Clipboard. Finally, an `ignore_memory_precedences` directive must be used after “go architect” in the Tcl file.

- Once you have finished all the build scripts, verify that the entire design can be synthesized without issues:

```
$ make clean
$ make
```

You should now have generated RTL! You can find the generated RTL in the file `dnn-accelerator-hls-unoptimized/build/Conv.v1/concat_rtl.v`.

- Finally, run an RTL simulation. You can get to the top level module by searching for module `Conv`, and then look inside it hierarchically.

When NCSim launches, several signals are added to the waveform window by default. Input and output ports for all the blocks are added by default, as well as several signals that indicate when the blocks in your design are active. Click through the hierarchy tab on the left as shown in Figure 5 to find the ProcessingElement module, and add the `ccs_ccore_en` signal of a PE to the waveform window. This signal is high whenever the PE is active. After you run the simulation, you can look at this signal to assess whether your design achieves high PE utilization. There might be other signals that would be helpful to look at. Please reference the Catapult User Reference to understand the names for the generated RTL. Note: Run a 4×4 design with a small test layer for faster debug iteration times.

Press the run button on the top to start the simulation as shown in Figure 5. The red `_error_cnt_sig`, which appears in the GUI after the top level ports, will tell you if there’s a mismatch between the C++ code and the generated Verilog. Verify that all the outputs match. The comparison result is also printed when you run the autograder. The RTL test compares the RTL outputs with C++ outputs. Therefore, if the RTL test shows no errors but the C++ test does, this does not mean that your design works.

- **If you make changes to your code and recompile, and see errors that you can not decipher, sometimes running `make clean` will help, before recompiling.**

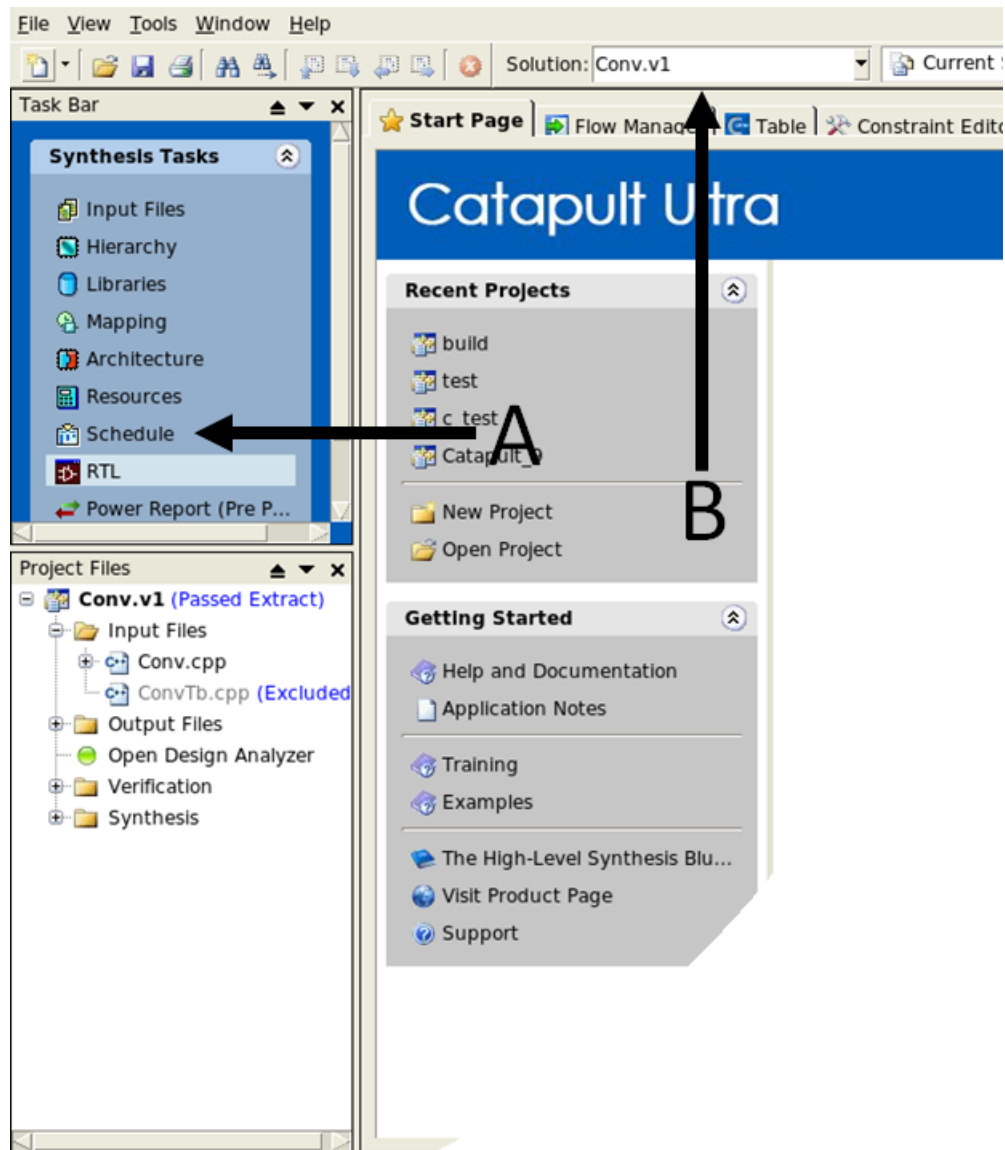


Figure 4: Click on (A) to switch to the Schedule step and click on the dropdown in (B) to switch between blocks.

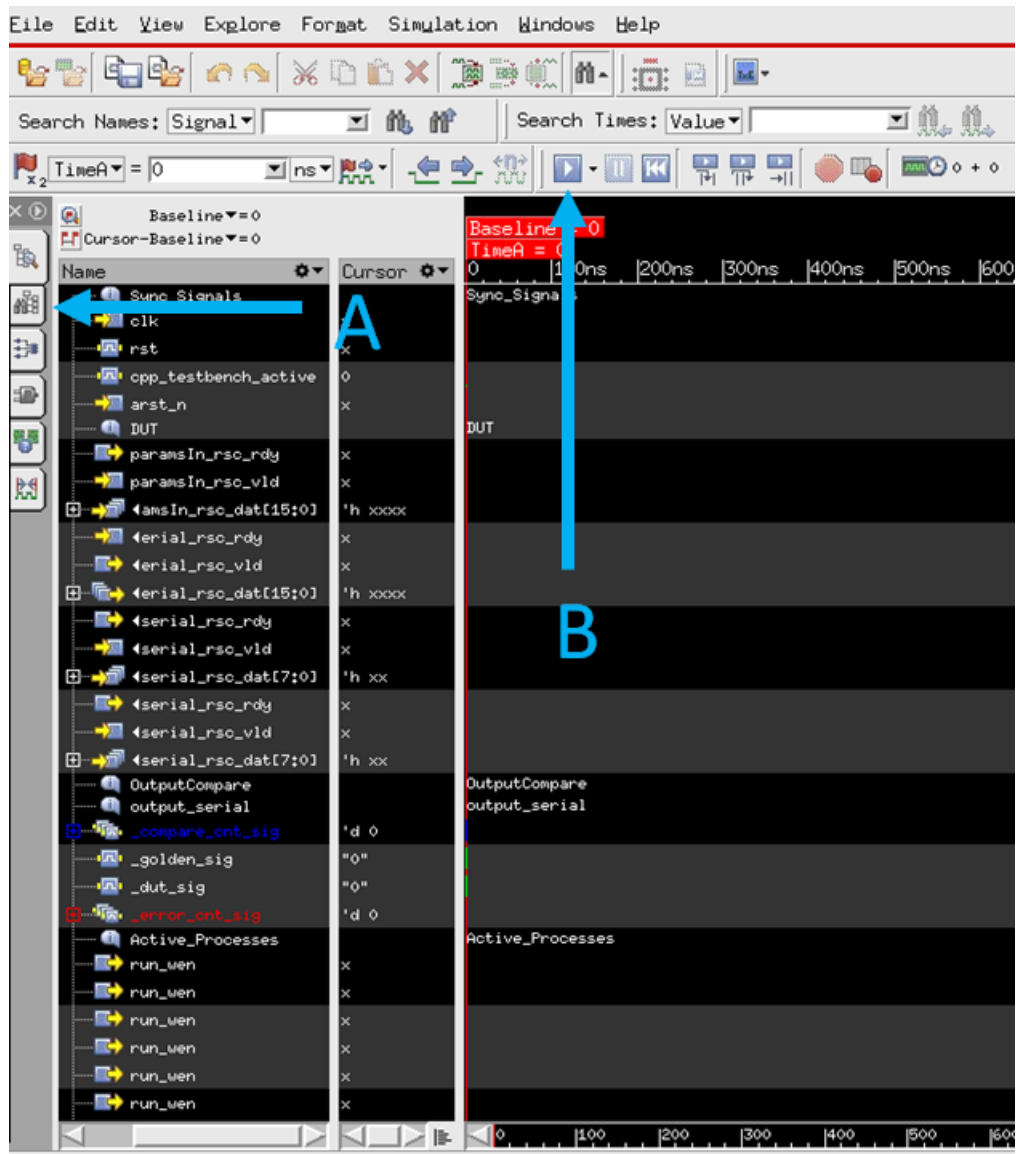


Figure 5: Click on the Hierarchy tab in (A) to go through the design hierarchy and find the `ccs_ccore_en` signal for a ProcessingElement. Click the start button in (B) to run the simulation.

```
$ make clean
```

8 Autograder

The autograder can help run the commands required to test your design. In this assignment, you must test both the functionality of your design in C and in Verilog.

To test in C, use the following command:

```
$ python3 autograder.py c_test -l layers/resnet_conv2_x_params.json
```

If you would like to run an RTL test, use the command:

```
$ python3 autograder.py rtl_test -l layers/small_layer1.json -v
```

If you want to debug the output of the commands that the autograder is running, use the verbose option `-v` or run the layer manually as indicated in the previous subsection.

Once your RTL is built, if you want to test a different layer without making any additional code changes, you can pass in `-n` to prevent rebuilding RTL. For example,

```
$ python3 autograder.py rtl_test -l layers/small_layer1.json -v
# And now you want to run another layer without rebuilding, you can do
$ python3 autograder.py rtl_test -l layers/small_layer2.json -v -n
```

Note if you made any change to your source change, you should not use this option.

Because without optimization your resultant RTL implementation will be very slow, we created 3 smaller layers for you to run RTL tests with. **To receive full credit, you will need to pass all ResNet-18 layers for `c_test`, but only `small_layer1`, `small_layer2`, and `small_layer3` in the `layers` folder for `rtl_test`.**

If you do not pass in any layer json file, the autograder will use `small_layer1.json`. You can pass in as many layers at a time as you want, just leave a space in between the JSON file names. Also, if you would like to test all the ResNet-18 layers or small layers at once, use the commands:

```
$ python3 autograder.py c_test -l all
$ python3 autograder.py rtl_test -l small
```

The RTL test command will set up your RTL simulation with the specified layers and run the test. The RTL test has to compile your HLS code to RTL and run a lengthy simulation, so this command will take a while.

Note that if you get an error in your RTL test, a useful way to debug is to look at the waveform. Use the `make rtl_test` command to pull up simulation waveform viewer and refer to the previous section on running the simulation.

If you need any help with the commands for the autograder use:

```
$ python3 autograder.py --help
```

9 Submission and Grading

9.1 Report

Submit a PDF report following the report template on Canvas.

9.2 Code

Please submit your code as a **.tar.gz** file to Gradescope. To pack things up, use:

```
$ tar -czvf filename.tar.gz /path/to/your/directory
```

You will **not** be graded for performance on this assignment. Feel free to start optimizing the design using what you have learned in class to get an early start on the next assignment. Please name your files using your and your partner's full names:

- `firstname1_lastname1-firstname2_lastname2.tar.gz`